# DefaultRadar

## A self updating machine learning system for loan default prediction

by Linga Reddy Gudisha

End to end MLOps project · Python, XGBoost, MLflow, FastAPI, Prefect, Evidently, Docker

### The project in one line

DefaultRadar predicts whether a loan applicant will repay or default, and then keeps itself healthy over time. The model is only a small part of it. The system builds its own data, trains the model, ships it, serves live predictions, watches the incoming data for changes, and retrains itself automatically when the world shifts. The whole thing runs on a laptop with a single command.

### The problem I wanted to solve

A bank decides whether to approve a loan using only what it knows the moment someone applies, like income, credit score, and how much they want to borrow. Predicting repayment is genuinely hard, but the real challenge is not the single prediction. A model placed into the real world slowly goes stale as customer behavior changes, so it has to be tracked, served, watched, and refreshed constantly. Most projects stop at the first guess in a notebook. I wanted to build the full picture that real teams actually run in production.

### How it works, the full lifecycle loop

**1 Features**
Clean the data with a strict guard that blocks any field the bank would only know after the loan.

→

**2 Train**
XGBoost learns to predict default, then calibrates its probabilities.

→

**3 Register**
Every version is saved with its score and full history.

→

**4 Gate**
Code, not a person, promotes a model only if it clears the quality bar.

→

**5 Serve**
A live service returns predictions and explains each one.

→

**6 Monitor**
Watches the data for drift and measures how far it shifted.

→

**7 Retrain**
On drift, it retrains, re checks the gate, and promotes the new model by itself.

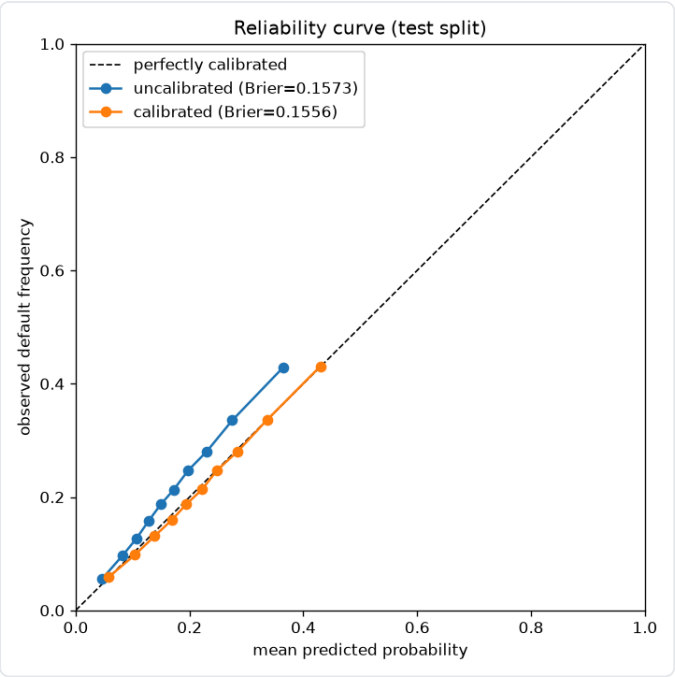
Step 7 loops back to step 2. That closing loop is the heart of the project. The system notices when the world changes and fixes itself, with no one touching it. That is exactly the work that real machine learning teams spend most of their time on, and it is rarely shown from start to finish.

### Keeping the numbers honest

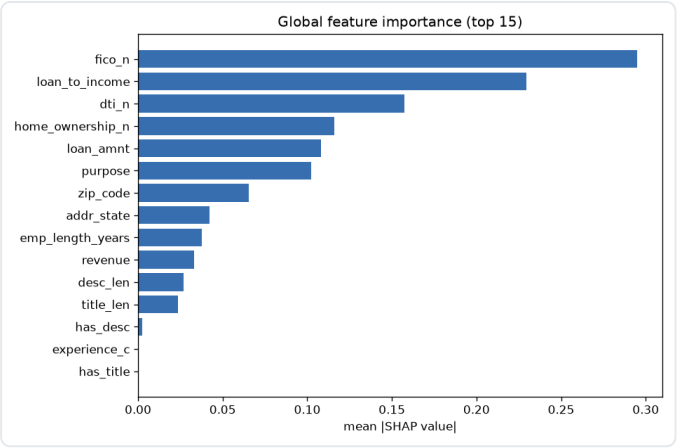
On strictly application time data, split by date so the model never sees the future, it scores about 0.68 on the standard ranking measure. That is a realistic number for this problem. I could have made it look better by letting the model peek at information it would never actually have, but I refused and I documented exactly why. I would rather show an honest result that holds up in production than a pretty one that quietly falls apart.

### Results at a glance

| | |
|---|---|
| **Honest test score (ranking)** | About 0.68 ROC AUC on a proper split by date, with no leakage |
| **Calibrated probabilities** | A score of 0.20 really means a 20 percent chance, verified against the raw model |
| **Prediction speed** | Under 15 milliseconds per request |
| **Self healing** | Detected data drift triggers an automatic retrain and promotion |
| **Engineering quality** | Full test suite plus automated checks that run on every code change |
| **One command to run** | docker compose up brings up the tracker, database, scorer, and scheduler together |




The model is calibrated so its probabilities can be trusted. The orange line sits close to the perfect line, and the score improves over the raw model.




What the model pays attention to. Credit score and the engineered loan to income ratio lead, which lines up with how a human would think.

## The technology I used

Python for everything, XGBoost and scikit learn for the model, MLflow for tracking and the model registry, FastAPI to serve live predictions, Prefect to run the monitoring and retraining flows, Evidently and PSI for drift detection, SHAP for explanations, DuckDB for fast analytics over the data, Docker to package the whole stack, and GitHub Actions for automated testing.

## Why this stands out

Most portfolios show a single notebook and a high accuracy number, which says very little about whether someone can ship. This project shows the part that real companies struggle with the most, taking a model from an idea all the way to a service that ships safely, explains itself, watches its own health, and recovers on its own when the data changes. That is the daily job of a machine learning engineer, and showing it end to end with honest numbers is rare.

Linga Reddy Gudisha

Code and write up: **github.com/lithin45/defaultradar**